Градиентный бустинг на деревьях решений

Что это такое, где используется и как выбрать библиотеку

Catboost XGBoost LightGBM

1. Что такое градиентный бустинг

- **Ансамблевый метод**: итоговая модель состоит из многих деревьев
- Деревья строятся **последовательно**, а не независимо
- Каждое новое дерево **исправляет ошибки** предыдущих
- **Итог**: сильная модель из набора **простых базовых моделей**

На табличных данных это один из самых сильных практических подходов.

2. Пример: как модели исправляют ошибки

Пример: прогноз цены квартиры

- **Дерево 1**: даёт грубую оценку по площади
- **Дерево 2**: замечает, что центр города систематически недооценён
- **Дерево 3**: добавляет поправку на ремонт

Было:

- базовый прогноз: `10.0 млн`

Поправки:

- район: `+2.0 млн`
- ремонт: `+0.8 млн`

Итог:

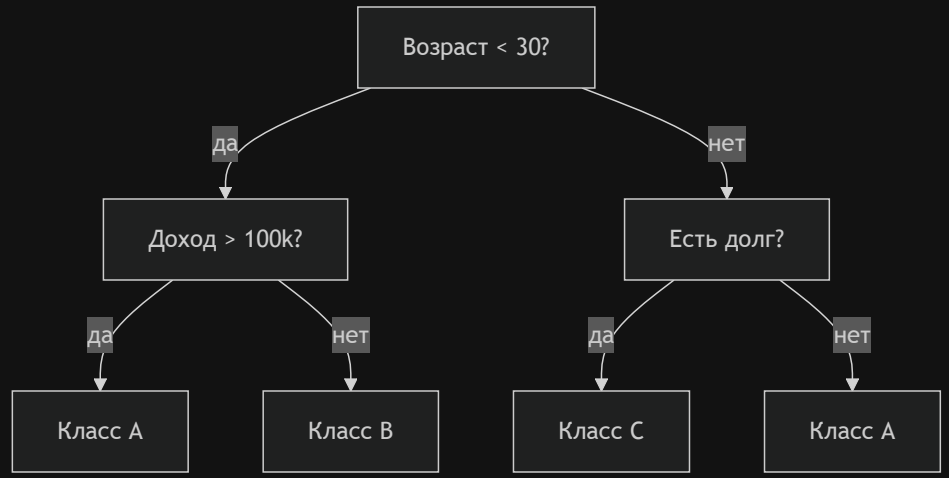
- `12.8 млн`

3. Почему именно деревья решений

- Хорошо работают с **табличными данными**
- Умеют ловить **нелинейности** и **взаимодействия признаков**
- Обычно не требуют сложного **масштабирования**
- Подходят как **слабые базовые модели**
- Часто устойчивы к «грязным» реальным данным

На практике это даёт:

- быстрый baseline
- меньше feature engineering
- хорошее качество в бизнес-задачах



4. Boosting vs одно дерево vs Random Forest

Подход	Как строятся модели	Идея
Одно дерево	Одна модель	Просто одно дерево
Bagging	Параллельно	Усреднение независимых моделей
Random Forest	Много независимых деревьев	Bagging для деревьев
Boosting	Последовательно	Новые деревья исправляют ошибки

- **Одно дерево** — просто и интерпретируемо, но нестабильно
- **Random Forest** — хорошо снижает разброс
- **Boosting** — часто даёт лучшее качество на таблицах

5. Где используется градиентный бустинг

Классификация
- скоринг
- антифрод
- churn prediction

Регрессия
- цена
- спрос
- выручка

Ранжирование
- поиск
- рекомендации
- ad ranking

- Часто показывает **очень сильные результаты на табличных данных**
- Один из первых кандидатов для **baseline** и production-модели

6. Общий принцип работы

Модель строится как сумма деревьев

$$F(x) = \sum_{m=1}^{M} \gamma_m h_m(x)$$

- $h_m(x)$ — очередное дерево
- γ_m — его вклад
- M — число итераций

7. Гиперпараметры и переобучение

Параметр	На что влияет
`learning_rate`	размер шага
`n_estimators` / `iterations`	число деревьев
`max_depth` / `depth`	сложность дерева
`num_leaves`	сложность дерева в LightGBM
`subsample`, `colsample_bytree`	регуляризация через подвыборки

Почему бывает overfitting

- слишком много деревьев
- слишком глубокие деревья
- большой learning rate

Как бороться

- early stopping
- уменьшать глубину
- уменьшать learning rate
- использовать subsample / colsample

8. XGBoost

Идея

- зрелая и эффективная реализация бустинга
- сильный и надёжный baseline

Плюсы

- стабильно высокое качество
- хорошая документация
- сильная регуляризация

Минусы

- категории часто требуют encoding
- на больших данных не всегда самый быстрый

Категориальные признаки

- чаще делают preprocessing

Скорость

- высокая

Когда выбирать

- нужен проверенный инструмент
- важна зрелость экосистемы
- нужен production baseline

8. XGBoost

```python
from xgboost import XGBClassifier
from sklearn.datasets import load_iris
from sklearn.model_selection import train_test_split

data = load_iris()
X_train, X_test, y_train, y_test = train_test_split(
  data['data'],
  data['target'],
  test_size=0.2,
)

model = XGBClassifier(
  n_estimators=2,
  max_depth=2,
  learning_rate=1,
  objective='binary:logistic',
)

model.fit(X_train, y_train)
preds = model.predict(X_test)
```

9. LightGBM

Идея

- быстрый бустинг для больших данных
- использует leaf-wise рост деревьев

Плюсы

- очень высокая скорость
- хорошая масштабируемость
- сильное качество

Минусы

- чувствителен к тюнингу
- может сильнее переобучаться

Категориальные признаки

- поддерживаются, но не так удобно, как в CatBoost

Скорость

- очень высокая

Когда выбирать

- большой датасет
- важно быстро обучаться
- есть время на настройку

9. LightGBM

```python
from lightgbm import LGBMClassifier
from sklearn.datasets import load_iris
from sklearn.model_selection import train_test_split

data = load_iris()
X_train, X_test, y_train, y_test = train_test_split(
  data['data'],
  data['target'],
  test_size=0.2,
)

model = LGBMClassifier(
    n_estimators=2,
    num_leaves=31,
    learning_rate=0.05,
)

model.fit(X_train, y_train)
preds = model.predict(X_test)
```

10. CatBoost

Идея

- бустинг, удобный для данных с категориями
- сильная встроенная работа с категориальными признаками

Плюсы

- часто хорошо работает из коробки
- меньше ручной подготовки
- сильный baseline для бизнес-таблиц

Минусы

- не всегда самый быстрый
- иногда тяжелее по ресурсам

Категориальные признаки

- главное преимущество CatBoost

Скорость

- хорошая

Когда выбирать

- много категориальных признаков
- нужен быстрый и сильный baseline
- хочется меньше feature engineering

10. CatBoost

```python
from catboost import CatBoostClassifier
from sklearn.datasets import load_iris
from sklearn.model_selection import train_test_split

data = load_iris()
X_train, X_test, y_train, y_test = train_test_split(
  data['data'],
  data['target'],
  test_size=0.2,
)

model = CatBoostClassifier(
    iterations=2,
    depth=6,
    learning_rate=0.05,
)

model.fit(X_train, y_train)
preds = model.predict(X_test)
```

11. Сравнение библиотек

Критерий	XGBoost	LightGBM	CatBoost
Скорость	высокая	очень высокая	средняя / высокая
Качество	высокое	очень высокое	очень высокое
Категории	чаще нужен encoding	поддержка есть	лучшая поддержка
Удобство	высокое	среднее / высокое	высокое
Тюнинг	средний	выше среднего	ниже среднего
Когда брать	надёжный baseline	большие данные	много категорий

Универсального победителя нет: лучший вариант зависит от данных и ограничений задачи.

12. Когда и что лучше брать?

CatBoost

- много категорий
- быстрый baseline из коробки

LightGBM

- большой датасет
- нужна высокая скорость

XGBoost

- нужен стабильный, проверенный инструмент
- важна зрелая экосистема

Когда бустинг не лучший выбор

- сырые изображения / текст / звук
- нужна максимально простая интерпретация
- нужны другие архитектуры, например DL

Запомнить

1. Бустинг — один из лучших методов для таблиц
2. Деревья исправляют ошибки друг друга
3. Выбор библиотеки зависит от данных

Спасибо за внимание!

Вопросы?